Unitree Week 38

Presentation by Mario Ortiz

This Week

- Upper Body Control and Onboard PC Access for Unitree (continued)
-

Unitree’s Official Arm Controller Summary

- ArmController(): connects to joint feedback and creates an arm-command publisher
- enable(): Requests SDK control and initializes joint targets
- set_q(): assigns desired positions to 13 joints
- movej(): interpolates all 13 joint targets
- release(): gradually transfers control back
-

Components we will change

- Add connection checks and feedback validation
- Implement a verified, gradual control handover
- Change set_q() so it only changes selected arm targets.
- movej(): Implement motion limits, trajectory validation and feedback monitoring

Current issues with the initial code to account for:

- enable() does not gradually provide ramp control as implied in the code. It requests full SDK authority, and should be fixed.
- Lack of bracket-specific safety restrictions: SDK does not check whether a trajectory would hit the bracket, or nearby objections. For use while connected to bracket, we must account for limitations of this sort.

First SDK Program

- We will start the program by reading the robot's current positions (similar to last week's r1_joint_monitor.cpp command).
- Additional components:
  - Validated position and velocity limits
  - Call-clearance checks
  - Control-mode verification.

Why we need a DDS controller

- Preset Action API: Executes existing gesture by ID. However, the API doesn’t offer a joint mask to exclude the head, waist, or lower body.
- Arm SDK DDS: Lets us provide our own joint trajectories through rt/arm_sdk. However, its command interface includes 13 joints, not just the 10 arm joints.

Unitree’s DDS C++ Code

- Initializes DDS Communication
- Subscribes to current motor positions
- Creates the motor-command publisher
- Requests SDK control authority

Custom R1 Diagnostic App

Features:

- Connects through “eth10”
- Reads FSM ID
- Reads FSM mode
- Displays any API errors (identifying firmware limitations)

R1 Arm Controller Initial

Joint Feedback Status:

- Tells us all the upper body joint angles.

```
unitree@ubuntu:~$ ~/r1_custom/build/r1_arm_controller --status
Waiting for R1 joint feedback...

--- Current R1 Upper-Body Joints ---
Left Shoulder Pitch: 1.19783 rad
Left Shoulder Roll: 0.67206 rad
Left Shoulder Yaw: -0.88543 rad
Left Elbow: 2.20568 rad
Left Wrist Roll: 1.37743 rad
Right Shoulder Pitch: 0.16993 rad
Right Shoulder Roll: 0.05286 rad
Right Shoulder Yaw: -0.15360 rad
Right Elbow: 0.99647 rad
Right Wrist Roll: 0.00233 rad
Waist Yaw: -0.08432 rad
Head Pitch: 0.00201 rad
Head Yaw: 2.00712 rad
unitree@ubuntu:~$ ~/r1_custom/build/r1_arm_controller --status
Waiting for R1 joint feedback...

--- Current R1 Upper-Body Joints ---
Left Shoulder Pitch: 0.03805 rad
Left Shoulder Roll: 0.28557 rad
Left Shoulder Yaw: 0.09817 rad
Left Elbow: -0.42663 rad
Left Wrist Roll: -1.59986 rad
Right Shoulder Pitch: 0.16992 rad
Right Shoulder Roll: 0.05317 rad
Right Shoulder Yaw: -0.15353 rad
Right Elbow: 0.99653 rad
Right Wrist Roll: 0.00229 rad
Waist Yaw: -0.08385 rad
Head Pitch: 0.00194 rad
Head Yaw: 2.00712 rad
```

R1 Initial Controller (Initial)

Gesture Waypoints

- Indicates specific joint angles that the R1 must reach to complete gestures (from the Unitree API)

```
unitree@ubuntu:~$ ~/r1_custom/build/r1_arm_controller --dry-run --gesture right_hand_up
Waiting for R1 joint feedback...

DRY RUN ONLY - NO MOTOR COMMANDS
Gesture prototype: right_hand_up
Targets are relative to current feedback.

Waypoint at t = 0.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.16991 rad
  Right Shoulder Yaw: -0.15351 -> -0.15351 rad
  Right Elbow: 0.99665 -> 0.99665 rad

Waypoint at t = 2.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.06991 rad
  Right Shoulder Yaw: -0.15351 -> -0.15351 rad
  Right Elbow: 0.99665 -> 1.07665 rad

Other 10 upper-body targets unchanged.
No leg targets generated.
No safety or collision validation performed.
No DDS motor publisher created.
```

R1 Initial Controller (Initial)

Wave Under Head Gesture Wavepoint

```
unitree@ubuntu:~$ ~/r1_custom/build/r1_arm_controller --dry-run --gesture wave_under_head
Waiting for R1 joint feedback...

DRY RUN ONLY - NO MOTOR COMMANDS
Gesture prototype: wave_under_head
Targets are relative to current feedback.

Waypoint at t = 0.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.16991 rad
  Right Shoulder Yaw: -0.15354 -> -0.15354 rad
  Right Elbow: 0.99665 -> 0.99665 rad

Waypoint at t = 1.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.06991 rad
  Right Shoulder Yaw: -0.15354 -> -0.15354 rad
  Right Elbow: 0.99665 -> 1.07665 rad

Waypoint at t = 2.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.06991 rad
  Right Shoulder Yaw: -0.15354 -> -0.10354 rad
  Right Elbow: 0.99665 -> 1.07665 rad

Waypoint at t = 3.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.06991 rad
  Right Shoulder Yaw: -0.15354 -> -0.20354 rad
  Right Elbow: 0.99665 -> 1.07665 rad

Waypoint at t = 4.00000 s
  Right Shoulder Pitch: 0.16991 -> 0.16991 rad
  Right Shoulder Yaw: -0.15354 -> -0.15354 rad
  Right Elbow: 0.99665 -> 0.99665 rad

Other 10 upper-body targets unchanged.
No leg targets generated.
No safety or collision validation performed.
No DDS motor publisher created.
unitree@ubuntu:~$
```

R1 Diagnostics

R1 Mode Diagnostic: Checks the FSM of the robot.

```
unitree@ubuntu:~$ ~/r1_custom/build/r1_mode_diagnostic
=== R1 EDU Mode Diagnostic ===
Network interface: eth10
READ-ONLY MODE

--- FSM ID ---
API return code: 0
Current FSM ID: 0
SDK interpretation: Zero Torque

--- FSM Mode ---
API return code: 3203
FSM mode unavailable or unsupported.

--- Safety Information ---
No mode changes requested.
No motor commands published.
No Arm SDK control requested.
FSM state alone does not establish safe arm-only operation.
unitree@ubuntu:~$
```

Last Steps

A SDK Control Script is being developed that does:

- First inputs tracked motion from the Unitree R1
- Then verifies R1 joint limitations
- Next command those same positions for a short interval, then release control, transmitting active SDK messages, in which the R1 could initiate the act.

References

- https://github.com/unitreerobotics/xr_teleoperate/blob/main/teleop/teleop_hand_and_arm.py
- https://github.com/unitreerobotics/xr_teleoperate/blob/main/teleop/robot_control/robot_arm.py
- https://github.com/unitreerobotics/unitree_sdk2_python/issues/182
- https://github.com/unitreerobotics/xr_teleoperate/wiki/Motion
- https://github.com/unitreerobotics/xr_teleoperate/issues/334
- https://marketing.unitree.com/article/en/R1_A5A7/User_Manual.html
- https://github.com/unitreerobotics/unitree_ros